БОЛЬШИЕ ВЫЗОВЫ 2025-2026

Agent AI

Разработка нейросетевой программной среды с динамической самомодификацией

НАПРАВЛЕНИЕ

Большие данные, ИИ
и информационная безопасность

РАЗРАБОТЧИК

Ролдугин Илья Владимирович
ГБПОУ ВО "ГПК"

Актуальность, Цели и Задачи

Актуальность

Индустрия переходит от простых чат-ботов к **автономным агентным системам**, способным самостоятельно решать сложные задачи.

Проблема

Существующие ИИ-клиенты имеют **статичную архитектуру**: их инструменты жестко заданы разработчиком и не меняются под задачу.

ЦЕЛЕВАЯ АУДИТОРИЯ

IT-специалисты и опытные пользователи

Цель проекта

Создание высокопроизводительного ИИ-агента с архитектурой **динамической самомодификации**.

КЛЮЧЕВЫЕ ЗАДАЧИ

- **Механизм Runtime-исполнения и модификации кода.**
- **Адаптивный UI (пресеты и HTML-вставки).**
- **Взаимодействие ИИ с GUI операционной системы.**
- **Автономное повышение качества кодинга модели.**

Ядро Системы и Базовые Инструменты

MAIN.PY

```python
from agent import Chat

ai = Chat()
ai.set_preset("developer")
ai.set_mode("file_structure", "sandbox")

ai.add_message(f"Задача пользователя: {input()}")
files = ai.ai_get(
    "Выведи список файлов для изменения",
    target_type=list,
    clear_history=False
)
ai.send("Выполни задачу, меняя файлы внутри песочницы")

# ...
```

Центральный узел

Класс **Chat** — это ядро агента. Он управляет API нейросети, хранит историю контекста, предоставляет доступ к инструментам и поддерживает локальное окружение (`self.local_env`).

БАЗОВЫЙ ИНСТРУМЕНТАРИЙ ОС

Инструмент Python

Исполнение сгенерированного кода напрямую в локальном окружении процесса.

Инструмент Shell

Интерактивная сессия терминала с сохранением контекста между вызовами.

Динамическая Самомодификация

АРХИТЕКТУРА ИЗОЛЯЦИИ (SANDBOX)

СЛОВАРЬ LOCAL_ENV

Экземпляр Chat
self

Переменная вывода result = ""

AGENT.PY (УПРОЩЕННАЯ РЕАЛИЗАЦИЯ ФРАГМЕНТА)

```
def python_tool(self, code):
    self.local_env["self"] = self
    self.local_env["result"] = ''
    exec(code, self.local_env, self.local_env)
    return str(self.local_env["result"])
```

ВЫЗОВ ИНСТРУМЕНТА PYTHON

```
import types

# 1. Определение логики инструмента
def custom_math_tool(self, expression):
    # Безопасный вызов eval без builtins
    res = eval(expression, {'__builtins__': {}})
    return f"Результат вычисления: {res}"

# 2. Привязка метода к инстансу Chat
self.custom_math_tool = types.MethodType(
    custom_math_tool, self
)

# 3. Регистрация JSON-схемы для Gemini
self.tools.append({
    # Описание в формате OpenAI API
})
```

Вспомогательные Инструменты Ядра

python_str

Специализированный инструмент форматирования. Принимает текст и возвращает его как Python-строку, **экранируя спецсимволы**. Критически важен для безопасной записи кода в файлы.

google_search

Интеграция с **Google Custom Search API**. Позволяет агенту самостоятельно искать актуальную информацию в интернете, компенсируя устаревание весов модели.

sandbox

Управление локальной песочницей. Копирует проект во временную папку и **запускает изолированный процесс сервера**. Позволяет агенту безопасно тестировать свои изменения.

http

Загружает веб-страницы по URL, **очищает HTML от тегов и скриптов** с помощью BeautifulSoup, возвращая агенту чистый, легко анализируемый текстовый контент.

user_profile

Система **долгосрочной памяти**. Сохраняет в JSON личные предпочтения пользователя, профессиональный контекст и общие инструкции, обеспечивая непрерывность между сессиями.

Система Пресетов, Промптов и ACL

ДИНАМИЧЕСКОЕ УПРАВЛЕНИЕ

Система пресетов позволяет **мгновенно перенастраивать агента** под задачу. При смене пресета меняются:

Системные промпты

Глобальные текстовые инструкции, задающие общую "личность", правила поведения и контекст (например, "Ты - Архитектор, планируй...").

Режимы

Добавляют свой промпт при включении, так же позволяют настроить ACL и запустить скрипт сбора данных.

Команды

Отправляют свой промпт при нажатии, так же позволяют запустить скрипт.

Доступ к инструментам

Например: можно заблокировать python.

Списки контроля доступа (ACL)

Жесткий перехват файловых операций на уровне Python (sys.addaudithook) для защиты проекта от разрушительных действий ИИ.

ПРИМЕРЫ ВСТРОЕННЫХ ПРЕСЕТОВ

«Безопасный»

Максимальная защита системы. Агенту **запрещено изменять любые файлы проекта** (ACL: только чтение). Запись разрешена исключительно в изолированные директории `temp/` и `sandbox/`.

«Разработчик»

Полный доступ к коду. Через *режимы* можно динамически включать исходники файлов в контекст по разделам (frontend, backend, plugins), чтобы агент **знал содержимое файлов, а не угадывал**.

«Ревьюер»

Жесткое ограничение на редактирование (ACL: Read-Only). Инициируется специальный промпт для **глубокого статического анализа качества кода**, поиска уязвимостей и архитектурных багов без риска поломки проекта.

Плагин web interface

ПОЛЬЗОВАТЕЛЬСКИЙ ОПЫТ

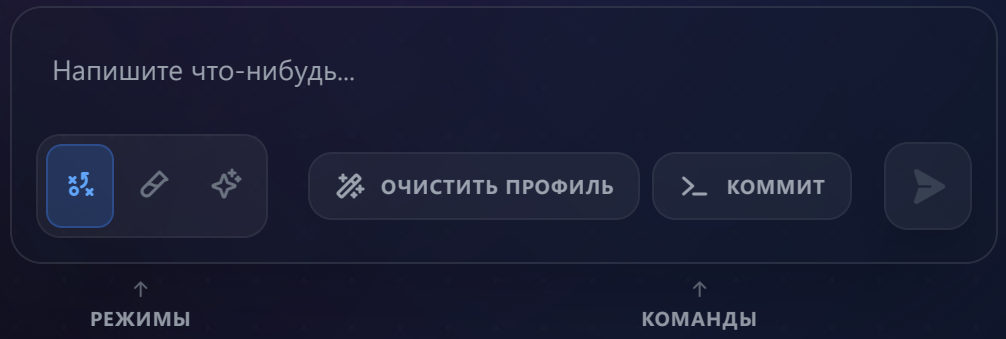
Полноценный Web-GUI

Современный интерфейс

Сервер (localhost) предоставляет адаптивный интерфейс на базе **Vue.js** и **Tailwind CSS**. Встроенные парсеры (Marked.js, Highlight.js, KaTeX) обеспечивают идеальное отображение кода, математики и markdown.

Сериализация сессий

Чаты сохраняются не просто как текст, а **сериализуются в бинарный формат (dill)**. Это позволяет сохранять сгенерированные в рантайме методы, переменные окружения и текущее состояние агента.

Визуальное управление

Через боковую панель можно **редактировать пресеты и промпты**, а управлять **режимами** и запускать **команды** прямо из панели ввода чата.